

Práctica 1

Variables Compartidas

1. Para el siguiente programa concurrente suponga que todas las variables están inicializadas en 0 antes de empezar. Indique cual/es de las siguientes opciones son verdaderas:

El ayudante de la práctica me dijo que no importaba mucho la cantidad de instrucciones en las que dividiera cada sentencia mientras se pudiera hacer lo que te pidiera el ejercicio. En este caso convenía “dividirlo” hasta las sumas nada mas.

(1.1)	$y := 4 * 2$
(1.2)	$x := y + 2$
(2.1)	$x := x + 1$
(3.1)	$(x * 3)$
(3.2)	$\dots + (x * 2)$
(3.3)	$x := \dots + 1$

A) Verdadera

Si se ejecutan secuencialmente los procesos, la X va a valer 56 al final
(1.1) -> (1.2) -> (2.1) -> (3.1) -> (3.2) -> (3.3)

B) Verdadera

Si se ejecutan las instrucciones en el siguiente orden:
(1.1) -> (3.1) -> (1.2) -> (3.2) -> (3.3) -> (2.1)

C) Verdadera

Si se ejecutan las instrucciones en el siguiente orden:
(1.1) -> (3.1) -> (1.2) -> (2.1) -> (3.2) -> (3.3)

2. Realice una solución concurrente de grano grueso (utilizando `<>` y/o `<await B; S>`) para el siguiente problema. Dado un número N verifique cuántas veces aparece ese número en un arreglo de longitud M . Escriba las pre-condiciones que considere necesarias

Precondiciones que asumo:

- El arreglo solo tiene enteros
- El número N aparece, como mínimo, una vez

Pseudocódigo

```
int[] arreglo = new int[N];
...
int N = ¿?
int cant_procesos = 4;
int veces = 0;

process Contador[id: 0..3]
{
    int veces_local = 0;
    int longitud = (N / cant_procesos) ;

    for (i = longitud * id; i < longitud; i++)
    {
        if (arreglo[i] == N)
            veces_local++;
    }

    <veces = veces + veces_local>
}
```

3. Dada la siguiente solución de grano grueso:

A) Indicar si el siguiente código funciona para resolver el problema de Productor/Consumidor con un buffer de tamaño N . En caso de no funcionar, debe hacer las modificaciones necesarias

No, el presente código no funciona correctamente ya que, al no proteger la línea que le sigue al `await` en cada uno de los dos procesos donde el buffer recibe o se guarda el elemento del buffer de la posición indicada, se corre el riesgo de que se guarde un elemento de una posición del buffer que es basura, o que se lea una posición que no corresponda.

Las modificaciones necesarias en este caso consisten en incluir a la sentencia inmediatamente superior en la *sección crítica*, para que así no ocurra interleaving en el medio de las dos sentencias.

```

Process Productor::
{ while (true)
  { produce elemento
    <await (cant < N); cant++
    buffer[pri_vacia] = elemento;>
    pri_vacia = (pri_vacia + 1) mod N;
  }
}

```

```

Process Consumidor::
{ while (true)
  { <await (cant > 0); cant-
    elemento = buffer[pri_ocupada];>
    pri_ocupada = (pri_ocupada + 1) mod N;
  }
}

```

B) Modificar el código para que funcione para C consumidores y P productores.

```

Process Productor::
{ while (true)
  { produce elemento
    <await (cant < N); cant++
    buffer[pri_vacia] = elemento;
    pri_vacia = (pri_vacia + 1) mod
N;>
  }
}

```

```

Process Consumidor::
{ while (true)
  { <await (cant > 0); cant-
    elemento = buffer[pri_ocupada];
    pri_ocupada = (pri_ocupada + 1) mod N;>
  }
}

```

Expandí la sección crítica para incluir además la suma del pri_vacia y el pri_ocupada

4. Resolver con SENTENCIAS AWAIT (<> y <await B; S>). Un sistema operativo mantiene 5 instancias de un recurso almacenadas en una cola, cuando un proceso necesita usar una instancia del recurso la saca de la cola, la usa y cuando termina de usarla la vuelve a depositar.

```
int cant = 5;
cola recursos = new cola[5];

process usarRecurso[1..N]
{
    <await (cant > 0); cant-;
    elem = cola.pop();>
    // Usar recurso
<cant++;
    cola.push(recurso);>
}
```